Index

a. theory question
b. MCQ question
c. Program or coding questions

Interview Question

Q21. Explain the difference between ArrayList and LinkedList?
Q22. Explain the stack class and relation of Vector and Stack?
Q23. is Stack thread safe or not if yes then why?
Q24. Explain the Set collection and why use Set collection ?
Q25. Explain types of Set collection with its methods?
Q26. Explain the HashSet in detail?
Q27. Explain the LinkedHashSet in detail?
Q28. Explain the TreeSet class in depth?
Q29. Explain the difference between HashSet, LinkedHashSet and TreeSet?
Q30. What is NavigableSet and why use it?
Q31. What is a collections class and why use it?
Q32. Explain the difference between Collection and Collections?
Q33. Explain the 5 methods of the Collections class?
Q34. Explain the Comparable and Comparator interface and why use it?
Q35. Explain the difference between Comparable and Comparator interface?
Q36. Explain how we can convert a synchronized collection or map into a synchronized collection?
Q37. What is a Map interface and why use Map with its methods ?
Q38. Explain types of Map?
Q39. Explain HashMap in detail?
Q40. Explain LinkedHashMap?

MCQ Questions

---

1. ArrayList vs LinkedList

Q1. What will be faster for frequent insertions in middle?
A) ArrayList
B) LinkedList
C) Both same
D) None

Answer: B
Explanation: LinkedList uses nodes → efficient insertion without shifting elements.

---

2. ArrayList vs LinkedList Output

Q2. What is the output?

```
List<Integer> list = new LinkedList<>();
list.add(10);
list.add(0, 5);
System.out.println(list);
```

A) [10,5]
B) [5,10]
C) Error
D) Null

Answer: B
Explanation: Index-based insertion at 0.

---

3. Stack Class

Q3. Output?

```
Stack<Integer> s = new Stack<>();
s.push(10);
s.push(20);
System.out.println(s.pop());
```

A) 10
B) 20
C) Error
D) Null

Answer: B
Explanation: Stack follows LIFO.

---

4. Stack Relation with Vector

Q4. Stack is subclass of which class?
A) ArrayList
B) LinkedList
C) Vector
D) HashSet

Answer: C
Explanation: Stack extends Vector.

---

5. Stack Thread Safety

Q5. Why is Stack thread-safe?
A) Uses Iterator
B) Uses synchronization
C) Uses Set
D) Uses Queue

Answer: B
Explanation: Inherited synchronized methods from Vector.

---

6. Set Behavior

Q6. Output?

```
Set<Integer> set = new HashSet<>();
set.add(1);
set.add(1);
System.out.println(set.size());
```

A) 0
B) 1
C) 2
D) Error

Answer: B
Explanation: No duplicates allowed.

---

7. LinkedHashSet Order

Q7. Output?

```
Set<Integer> set = new LinkedHashSet<>();
set.add(3);
set.add(1);
set.add(2);
System.out.println(set);
```

A) [1,2,3]
B) [3,1,2]
C) Random
D) Error

Answer: B
Explanation: Maintains insertion order.

---

8. TreeSet Sorting

Q8. Output?

```
Set<Integer> set = new TreeSet<>();
set.add(3);
set.add(1);
set.add(2);
System.out.println(set);
```

A) [3,1,2]
B) [1,2,3]
C) Random
D) Error

Answer: B
Explanation: TreeSet sorts elements.

---

9. TreeSet Null Handling

Q9. What happens?

```
Set<Integer> set = new TreeSet<>();
set.add(null);
```

A) Works
B) Runtime error
C) Compile error
D) Null stored

Answer: B
Explanation: TreeSet does not allow null.

---

10. HashSet Null Handling

Q10. Output?

```
Set<Integer> set = new HashSet<>();
set.add(null);
System.out.println(set);
```

A) Error
B) [null]
C) []
D) NullPointer

Answer: B
Explanation: HashSet allows one null.

---

11. NavigableSet

Q11. Output?

```
NavigableSet<Integer> set = new TreeSet<>();
set.add(10);
set.add(20);
System.out.println(set.lower(20));
```

A) 20
B) 10
C) null
D) Error

Answer: B
Explanation: lower() returns element less than given.

---

12. Collections Class

Q12. Output?

```
List<Integer> list = new ArrayList<>(List.of(3,1,2));
Collections.sort(list);
System.out.println(list);
```

A) [3,1,2]
B) [1,2,3]
C) Error
D) Null

Answer: B

---

13. Collection vs Collections

Q13. Which is true?
A) Collection is class
B) Collections is interface
C) Collections provides utility methods
D) Both same

Answer: C

---

14. Collections.reverse()

Q14. Output?

```
List<Integer> list = new ArrayList<>(List.of(1,2,3));
Collections.reverse(list);
System.out.println(list);
```

A) [1,2,3]
B) [3,2,1]
C) Error
D) Null

Answer: B

---

15. Comparable Interface

Q15. What does Comparable define?
A) Multiple sorting
B) Natural sorting
C) Random sorting
D) No sorting

Answer: B

---

16. Comparator Interface

Q16. What is Comparator used for?
A) Default sorting

B) Custom sorting
C) Thread safety
D) Searching

Answer: B

---

17. Comparable vs Comparator

Q17. Which supports multiple sorting logic?
A) Comparable
B) Comparator
C) Both
D) None

Answer: B

---

18. Synchronized Collection

Q18. Which method makes list synchronized?
A) Collections.sort()
B) Collections.reverse()
C) Collections.synchronizedList()
D) List.sync()

Answer: C

---

19. HashMap

Q19. Output?

```
Map<Integer,String> map = new HashMap<>();
map.put(1,"A");
map.put(1,"B");
System.out.println(map);
```

A) {1=A}
B) {1=B}
C) {1=A,1=B}
D) Error

Answer: B
Explanation: Duplicate key replaces value.

---

20. LinkedHashMap

Q20. Output?

```
Map<Integer,String> map = new LinkedHashMap<>();
map.put(2,"B");
map.put(1,"A");
System.out.println(map);
```

A) {1=A,2=B}
B) {2=B,1=A}
C) Random
D) Error

Answer: B
Explanation: Maintains insertion order.

Coding Questions

---

1. Sort Students by Marks (Comparable)

Problem: Create a Student object (id, name, marks) and sort students in ascending order of marks using natural ordering.
Example:
Input: [70, 85, 60]
Output: [60, 70, 85]

---

2. Sort Students by Name (Comparator)

Problem: Sort Student objects alphabetically by name using a custom comparator.
Example:
Input: [Rahul, Amit, Neha]
Output: [Amit, Neha, Rahul]

---

3. Sort Employees by Salary then Name

Problem: Sort Employee objects first by salary (ascending), if equal then by name.
Example:
Input: [(A,5000),(B,4000),(C,5000)]
Output: [(B,4000),(A,5000),(C,5000)]

---

4. Sort Products by Price Descending

Problem: Sort product objects based on price in descending order.
Example:
Input: [100, 500, 200]
Output: [500, 200, 100]

---

5. Sort Students by Multiple Criteria

Problem: Sort students by marks (descending) and age (ascending).
Example:
Input: [(marks=90,age=20),(marks=90,age=18)]
Output: [(90,18),(90,20)]

---

6. Detect Duplicate Objects using Set

Problem: Store Student objects and detect duplicates based on id.
Example:
Input: IDs [101,102,101]
Output: Duplicate → 101

---

7. Maintain Insertion Order with Objects

Problem: Store objects in a structure that removes duplicates but preserves insertion order.
Example:
Input: [A,B,A,C]
Output: [A,B,C]

◆ ⍰ Part B: Map + String / Array Problems

---

8. Count Character Frequency

Problem: Count frequency of each character in a string.
Example:
Input: "aabbc"
Output: {a=2, b=2, c=1}

---

9. Find First Non-Repeating Character

Problem: Find the first character that does not repeat in a string.
Example:
Input: "aabbcdde"
Output: c

---

10. Find Duplicate Characters

Problem: Print all duplicate characters in a string.
Example:
Input: "programming"
Output: r, g, m

---

11. Count Words in Sentence

Problem: Count frequency of each word using a map.
Example:
Input: "java is java easy"
Output: {java=2, is=1, easy=1}

---

12. Find Unique Elements in Integer Array

Problem: Extract elements that appear only once.
Example:
Input: [1,2,2,3,4,4]
Output: [1,3]

---

13. Find Duplicate Elements in Array

Problem: Identify all duplicate numbers in an array.
Example:
Input: [1,2,3,2,4,1]
Output: [1,2]

---

14. Check Anagram Using Map

Problem: Check if two strings are anagrams using frequency logic.
Example:
Input: "listen", "silent"
Output: True

---

15. Group Anagrams

Problem: Group words that are anagrams of each other.
Example:
Input: ["eat","tea","tan","ate"]
Output: [["eat","tea","ate"],["tan"]]

◆ ● Part C: Map + Collection Combination Problems

---

16. Sort Map by Keys

Problem: Sort map entries based on keys.
Example:
Input: {3=A,1=B,2=C}
Output: {1=B,2=C,3=A}

---

17. Sort Map by Values

Problem: Sort map entries based on values.
Example:
Input: {1=Z,2=A,3=M}
Output: {2=A,3=M,1=Z}

---

18. Find Top K Frequent Elements

Problem: Find K most frequent elements using map + sorting.
Example:
Input: [1,1,1,2,2,3], k=2
Output: [1,2]

---

19. Find Pair with Given Sum

Problem: Find pairs in an array whose sum equals a target using map.
Example:
Input: [2,7,11,15], target=9
Output: (2,7)

---

20. Longest Substring Without Repeating Characters

Problem: Find the length of longest substring without repeating characters.
Example:
Input: "abcabcbb"
Output: 3